Topos — Documentation

> **One combined file** for all user-facing Topos documentation: the mental model, a runnable walkthrough, the full API reference, and usage patterns. The engineering record (`SPECIFICATION.md` , `DECISIONS.md`) lives separately — this is the user manual.
>
> **Status:** M0–M6 implemented · M7 (spectral) deferred by design · M8 API-stability scope done · M9 implemented · M10 (MCP server) implemented · M11 phase 1 implemented (Centrality/PageRank/ Directed SCC, GDS-verified against a live instance; phase 2 — s-connected-components/s-line-graph/ s-diameter — not yet scheduled). 233 tests pass. License: **MIT** (decided 2026-07-26). NuGet packages (`Topos.Hypergraph` , `Topos.Hypergraph.Persistence` , `Topos.Hypergraph.Knowledge`) are published — see §1 Installation for the current state.

Table of contents

- Overview
- 1. Installation
- 1.1 From NuGet (recommended, once published)
- 1.2 From source (works today)
- 2. Concepts — the mental model
- 2.1 What Topos is
- 2.2 The four primitives
- 2.3 The two invariants
- 2.4 Roles vs. VertexRoles — the single most confusing fork
- 2.5 Hyperedges as reified vertices
- 2.6 The layer architecture
- 2.7 Concurrency
- 2.8 What Topos is not
- 3. Getting started
- 3.1 Create a kernel and some vertices
- 3.2 Define typed properties
- 3.3 Build an n-ary hyperedge
- 3.4 Query it back
- 3.5 Save and reload
- 4. API reference
- 4.1 Kernel — `Topos.Hypergraph`
- 4.2 Reification & semantics

- 4.3 Views & set algebra
- 4.4 Embeddings & learnable edges
- 4.5 Analytics
- 4.6 Persistence — `Topos.Hypergraph.Persistence`
- 4.7 Knowledge — `Topos.Hypergraph.Knowledge`
- 4.8 Internal types — not public API
- 5. Usage patterns
- 5.1 N-ary facts as one hyperedge
- 5.2 Reification: edges as vertices
- 5.3 Per-membership ("cell") data without a kernel primitive
- 5.4 Composable views
- 5.5 Semantic recall (vector search)
- 5.6 Learnable edges
- 5.7 Persistence (save / reload)
- 5.8 Directed (role-aware) traversal
- 5.9 Ranking and structural analysis (centrality, PageRank, directed SCC)
- 6. MCP server — agent tool-calling
- 6.1 What it is
- 6.2 Wire it into an agent
- 6.3 The 18 tools
- 6.4 A worked example: agent builds and queries a hyperedge
- 6.5 What's deliberately not exposed
- 6.6 State lifetime and limitations

Overview

Topos is an **embedded, in-process, typed-property hypergraph library for C#**, purpose-fit for AI / agent memory — LLM reasoning, explainability, learnable edges, tiered memory, and provenance. `[verified:docs=docs/SPECIFICATION.md §2.1]`

Topos (Greek τόπος, "place / location") is the root of *topology*. The name invokes the central thesis this library serves: **knowledge stored as topological graph structure rather than neural-network weights.** "Embedded, in-process" means like SQLite or Kuzu: you reference it as a library and hold a `HypergraphKernel` object in your process, not a server you connect to. "Hypergraph" means an edge (a *hyperedge*) can have **any number of member vertices**, not just two — which is the whole reason this library exists instead of just using a property graph.

Three packages: `[verified:src=src/Topos.Hypergraph/Topos.Hypergraph.csproj]` `[verified:src=src/Topos.Hypergraph.Persistence/Topos.Hypergraph.Persistence.csproj]` `[verified:src=src/Topos.Hypergraph.Knowledge/Topos.Hypergraph.Knowledge.csproj]` `[verified:src=src/Topos.Hypergraph.Mcp/Topos.Hypergraph.Mcp.csproj]`

Package	What it's for	Required?
`Topos.Hypergraph`	The kernel + algorithms (storage contract, BFS/DFS, views, embeddings, learnable edges, analytics).	**Yes** — everything depends on this.
`Topos.Hypergraph.Persistence`	Save/load a kernel to/from disk (`HypergraphSnapshot`).	Only if you need persistence.
`Topos.Hypergraph.Knowledge`	Layer-1 role-aware directed traversal (`DirectedBfs` , `RoleFilteredMembers` , `AddIncidence<TRole>`).	Only if your domain has directed/role-gated semantics.
`Topos.Hypergraph.Mcp`	A Model Context Protocol server exposing the kernel's API as 18 agent-callable tools (M10). Lets any MCP-aware agent (Claude Code, Cursor, Continue, ZCode) create vertices, build n-ary hyperedges, and query traversals without writing C#.	Only if you want agents to drive Topos. See §6.

---

1. Installation

Topos targets **.NET 10** (`net10.0`) with `Nullable` + `ImplicitUsings` enabled. `[verified:src=src/Topos.Hypergraph/Topos.Hypergraph.csproj]`

1.1 From NuGet (recommended, once published)

The three packages are being prepped for first publish under the **MIT** license (decided 2026-07-26). Once live, install with:

```
dotnet add package Topos.Hypergraph                 # required — kernel + algorithms
dotnet add package Topos.Hypergraph.Persistence     # optional — save/load
dotnet add package Topos.Hypergraph.Knowledge       # optional — directed/role-aware traversal
```

The first release will be the milestone prerelease versions: `Topos.Hypergraph` `0.1.0-m8` , `Topos.Hypergraph.Persistence` `0.1.0-m8` , `Topos.Hypergraph.Knowledge` `0.1.0-m9` . As prereleases, install opts in with `--prerelease` : `[verified:src=src/Topos.Hypergraph/Topos.Hypergraph.csproj — <Version>0.1.0-m8</Version>]`

```
dotnet add package Topos.Hypergraph --prerelease
```

> **Status note:** the NuGet publish is the gated item from `docs/DECISIONS.md`'s "M8 CLOSED" entry; the license decision (MIT) was made 2026-07-26 and the publish steps are documented in `docs/NUGET_PUBLISH_CHECKLIST.md`. Until the packages are live, use the source build below.

1.2 From source (works today)

If the NuGet packages aren't yet published — or you want to track `main` — reference Topos from source via a `ProjectReference`:

```
<!-- in your consuming .csproj -->
<ItemGroup>
  <ProjectReference Include="path/to/Topos/src/Topos.Hypergraph/Topos.Hypergraph.csproj" />
  <!-- add Topos.Hypergraph.Persistence for save/load -->
  <!-- add Topos.Hypergraph.Knowledge for directed/role-aware traversal -->
</ItemGroup>
```

Verify your environment against the whole solution: `[verified:src=Topos.sln]`

```
dotnet build Topos.sln
dotnet test Topos.sln
```

---

2. Concepts — the mental model

2.1 What Topos is

Topos is an **embedded, in-process, typed-property hypergraph kernel for C#**, optimized for long-lived adaptive symbolic memory workloads — incremental updates, provenance, explainability, retrieval, symbolic+vector coexistence, stable identities, partial activation, mutable knowledge that grows rather than being overwritten. `[verified:docs=docs/SPECIFICATION.md §2.1]`

The full thesis and the workload argument for it live in `docs/SPECIFICATION.md §1`; this section is about *how the thing is shaped*, not *why it should exist*.

2.2 The four primitives

Everything in Topos is built from four primitive types — the load-bearing storage contract; spec §3 pressure-tested them across four review rounds and they survived every attempt to add a fifth.
`[verified:docs=docs/SPECIFICATION.md §3]` `[verified:docs=docs/DECISIONS.md §1]`

2.2.1 `Handle` — stable identity

```
public readonly record struct Handle(uint Index, uint Generation = 0);
// src/Topos.Hypergraph/Handle.cs
```

A `Handle` is the stable identity of a vertex. `Index` is a **monotonic, never-reused counter** — once a vertex exists, its Handle's logical identity never changes and is never recycled, even after the vertex goes dormant. `[verified:src=src/Topos.Hypergraph/Handle.cs:6-11]`

`Generation` is a reserved field for a possible future physical-slot-compaction feature; it is always `0` today and load-bearing for nothing. The field exists so the struct layout is stable if that future milestone ever needs it. `[verified:src=src/Topos.Hypergraph/Handle.cs:11-15]`

Two sentinels worth knowing: `Handle.Invalid` (the "no vertex" marker, `new(uint.MaxValue, uint.MaxValue)`) and `Handle.IsValid` (a convenience check). Failure paths on `TryGetVertex` set the out `Vertex`'s `Handle` to `Invalid` rather than C#'s `default(Handle)` — which would otherwise be indistinguishable from a real vertex #0. Always check the `bool` returned by `TryGetVertex`; `IsValid` is defense-in-depth, not the primary contract. `[verified:src=src/Topos.Hypergraph/Handle.cs:19-36]`

2.2.2 `Vertex` — the node record

```
public readonly record struct Vertex(Handle Handle, VertexRoles Roles, VertexStatus Status)
{
    public bool IsDormant => Status == VertexStatus.Dormant;
}
// src/Topos.Hypergraph/Vertex.cs
```

A `Vertex` is the node record. Notice what is **not** here: no data field, no dictionary of properties. Typed data attaches separately via `PropertyKey<T>` pools (primitive 4 below). The only things on the record itself are `Handle`, `Roles`, and `Status` — and those three sit inline on the record *because they are read on every traversal hop*; a property-bag indirection in that inner loop would be the wrong tier. `[verified:src=src/Topos.Hypergraph/Vertex.cs:3-8]`

- **`VertexRoles`** (`[Flags] enum : byte`) — kernel-level roles only. Today there is exactly one: `VertexRoles.Edge`, marking a vertex as a *reified hyperedge* (see §2.5). Domain-specific roles do **not** live here — see §2.4. `[verified:src=src/Topos.Hypergraph/VertexRoles.cs]`
- **`VertexStatus`** (`enum : byte`) — `Active` or `Dormant`. Read per-hop to skip dormant vertices during traversal. `[verified:src=Topos.Hypergraph/VertexStatus.cs]`

2.2.3 `Incidence` — one membership

```
public readonly record struct Incidence(Handle Source, Handle Member, byte Role, int Ordinal);
// src/Topos.Hypergraph/Incidence.cs
```

An `Incidence` is **one membership** in a hyperedge: `Member` participates in `Source` under `Role` at position `Ordinal`. This is the primitive that makes the hypergraph n-ary: one hyperedge is just the collection of all incidences sharing a `Source`. `[verified:src=src/Topos.Hypergraph/Incidence.cs:3–5]`

- **`Source`** is the Handle of a vertex tagged `VertexRoles.Edge`.
- **`Member`** is a participant vertex.
- **`Role`** is a raw `byte`. The kernel does not interpret it. Domain meaning lives in your layer-1 code (see §2.4). The recommended way to define role bytes is a `byte`-backed `enum` — see `docs/ROLE_CONVENTIONS.md`.
- **`Ordinal`** is an `int` position. Useful when order matters (e.g. "the Anchor is ordinal 0, the Target is the last member").

2.2.4 `PropertyKey<T>` — typed data identity

```
public readonly record struct PropertyKey<T>
{
    internal PropertyKey(string name, int id) { Name = name; Id = id; }
    public string Name { get; }
    public int Id { get; }
}
// src/Topos.Hypergraph/PropertyKey.cs
```

A `PropertyKey<T>` is the *typed identity* of a property, separate from its storage. `Name` is the stable, human-facing identity (e.g. `"content"`, `"embedding"`); `Id` is a per-process integer slot resolved once via `PropertyRegistry` and cached on the key for O(1) lookup. You do **not** construct one directly — the constructor is `internal` (locked M8); the only way to get one is `kernel.ResolveProperty<T>("name")`.
`[verified:src=src/Topos.Hypergraph/PropertyKey.cs:6–26]`

Behind the scenes, each `(T)` lives in its own columnar pool — an EnTT-style sparse-set, where every vertex with a value for that property sits in a dense parallel array. Columnar on purpose: iterating "every value of property X" (the shape persistence and analytics want) is a tight loop over one array, not a scatter-read across vertices.
`[verified:docs=docs/SPECIFICATION.md §3]`

2.3 The two invariants

The four primitives are governed by two invariants. These are not conventions — they are rules the kernel enforces, and your code can rely on them. `[verified:docs=docs/SPECIFICATION.md §3]`

Invariant 1 — dormant is never garbage-collected

> A vertex, once created, is never removed. Going `Dormant` tombstones it (it stops participating in new traversals) but it stays resolvable forever — including as a `Member` target of an `Incidence`.

The operational consequence: **provenance edges always resolve.** If you record "fact F was derived from fact G," and later mark G dormant, the edge from F to G still resolves to G's `Vertex`. You never get a dangling Handle. `[verified:src=src/Topos.Hypergraph/Incidence.cs:16-28]`

This is why `Handle.Index` is never reused: if it were, a stale Handle from an old provenance edge could silently point at a *different* vertex after recycling. Monotonic allocation + dormant-never-GC together make Handle identity a safe long-lived reference. `[verified:src=src/Topos.Hypergraph/HandleAllocator.cs:4-7]`

Invariant 2 — `VertexRoles` and `Incidence.Role` are independent axes

A vertex has a kernel-level `VertexRoles` (today: `None` or `Edge`). A vertex's *participation* in a hyperedge has a layer-level `Incidence.Role` byte (your domain's meaning). **These do not interact.** A vertex tagged `VertexRoles.Edge` (itself a reified hyperedge) can participate as a member of another hyperedge under any role byte — that's how nested reification works (see §5.2). `[verified:docs=docs/SPECIFICATION.md §3]`

2.4 Roles vs. `VertexRoles` — the single most confusing fork

If you read one section twice, read this one. There are two completely separate "role" concepts and they share a name:

Concept	Type	Where	Who interprets it
Kernel role	`VertexRoles` (`[Flags] enum : byte`)	inline on `Vertex`	the kernel; `Edge` is the only member today
Domain role	`byte` (the `Role` field of `Incidence`)	on each `Incidence`	**your layer-1 code, never the kernel**

The kernel-level `VertexRoles.Edge` marks "this vertex is itself a reified hyperedge." That's it. The kernel has no concept of Anchor, Condition, Target, Speaker, Mentioned, Before, After — those are all *domain* roles, carried as the raw `byte` on each `Incidence`, and interpreted by the layer sitting on top of the kernel.

This split is the "**the kernel records; it does not judge**" principle from spec §4.1. The kernel stores role bytes faithfully and indexes them for O(1) lookup; it does not validate cardinalities (e.g. "exactly one Anchor") or attach semantics to specific byte values. Cardinality validation, role-naming conventions, and any business rule about what a role means all live in layer-1 — either in your consumer code or in the optional `Topos.Hypergraph.Knowledge` package for the patterns multiple consumers have already needed. `[verified:src=src/Topos.Hypergraph/Incidence.cs:6-15]` `[verified:docs=docs/SPECIFICATION.md §4.1]`

The recommended pattern for defining your domain's role bytes is a `byte`-backed `enum` — see `docs/ROLE_CONVENTIONS.md`.

2.5 Hyperedges as reified vertices

A hyperedge in Topos is **not a separate primitive**. It is a vertex tagged `VertexRoles.Edge`, connected to its members by incidences where `Source` is the edge-vertex's Handle. This is the "Role:Edge reification" pattern (spec §7 pattern 12). `[verified:docs=docs/SPECIFICATION.md §7]`

Concretely, to model "one conversation turn mentioning three entities" as a single n-ary relationship:

```
var turn     = kernel.CreateVertex();                 // a domain vertex
var kyoto    = kernel.CreateVertex();                 // a domain vertex
var nara     = kernel.CreateVertex();
var osaka    = kernel.CreateVertex();

var mention  = kernel.CreateVertex(VertexRoles.Edge);  // the hyperedge, reified as a vertex
kernel.AddIncidence(mention, turn,  (byte)Role.Speaker,  ordinal: 0);
kernel.AddIncidence(mention, kyoto, (byte)Role.Entity,   ordinal: 1);
kernel.AddIncidence(mention, nara,  (byte)Role.Entity,   ordinal: 2);
kernel.AddIncidence(mention, osaka, (byte)Role.Entity,   ordinal: 3);
```

One hyperedge, four members — not three separate binary edges. This is the concrete shape of spec §1's thesis: a single utterance mentioning N things together is one atomic event, and the storage substrate preserves that atomicity rather than fragmenting it. `[verified:src=samples/Topos.Samples.ChatMemory/ChatMemory.cs:71-79]`

Why reify edges as vertices, instead of having a separate edge concept? Because it makes edges first-class: an edge can have properties (`SetProperty` works on any Handle, including an edge-vertex's), it can participate in *other* hyperedges (nested reification — see §5.2), and it can be the target of a provenance edge. None of that needs a special case.

2.6 The layer architecture

Topos is a 3-layer substrate: a **Storage model** (the kernel primitives), a **Graph model** (the algorithms over those primitives), and a **Knowledge model** (domain semantics). This is locked (spec §4), and the namespace boundaries in the shipped packages mirror it. `[verified:docs=docs/SPECIFICATION.md §4]`

```
┌──────────────────────────────────────────────────────────┐
│  Layer 1 — Knowledge model (domain semantics)            │
│  Your code, optionally + Topos.Hypergraph.Knowledge (M9) │
│  Role meanings, cardinality rules, directed/role-gated walk │
├──────────────────────────────────────────────────────────┤
│  Layer 2 — Graph model (algorithms)                      │
│  Topos.Hypergraph: IHypergraphQuery + BFS/DFS/shortest-  │
│  path/cycle/components/s-walk/community detection        │
├──────────────────────────────────────────────────────────┤
│  Layer 3 — Storage model (the kernel)                    │
│  Topos.Hypergraph: Handle / Vertex / Incidence /         │
│  PropertyKey<T>, the 2 invariants, SWMR concurrency      │
└──────────────────────────────────────────────────────────┘
```

The key discipline: **role-aware / directed traversal stays out of layer 2.** Every default algorithm on `IHypergraphQuery` (`GetBfs`, `GetDfs`, `GetShortestPath`, `GetConnectedComponents`, `HasCycle`) is **role-blind and co-membership-symmetric** — "the kernel does not judge" (spec §4.1). They treat any two vertices co-incident on the same hyperedge as adjacent, with no notion of "Anchor→Target direction." `[verified:src=src/Topos.Hypergraph/IHypergraphQuery.cs:44–56]`

That judgment lives in **layer 1** — either in your consumer code (a ~10-line role-filtered walk, which three independent consumers have hand-rolled) or, since M9, in the optional `Topos.Hypergraph.Knowledge` package, which offers `DirectedBfs` / `DirectedShortestPath` / `RoleFilteredMembers` over the same `IHypergraphQuery` surface. See §4.7.
`[verified:src=src/Topos.Hypergraph.Knowledge/DirectedTraversal.cs:10–16]`

Persistence (`Topos.Hypergraph.Persistence`) is a separate package that sits across these layers — it serializes layer 3's storage to disk and back.

2.7 Concurrency

The kernel uses a **Single-Writer / Multi-Reader (SWMR)** model. Handle allocation is genuinely lock-free (`Interlocked.Increment`); every other piece of state — the vertex table, the incidence indexes, every property pool — is one sparse-set behind its own `ReaderWriterLockSlim`. Read methods are always safe to call concurrently with the single writer; write methods assume a single-writer thread (concurrent writes need external synchronization). This is a deliberate, benchmark-driven correction: the original copy-on-write design measured 5–6× slower than naive and O(N²) on hub vertices; the per-pool-lock design measured *faster* than naive and eliminated the pathology. Full data in `docs/M0_BENCHMARK_RESULTS_2026-07-24.md`.
`[verified:src=src/Topos.Hypergraph/HypergraphKernel.cs:6–31]` `[verified:docs=docs/DECISIONS.md §3.4]`

2.8 What Topos is not

Scope boundaries, to prevent creeping expectations (lifted from spec §2.2):

- **Not an extraction pipeline.** Topos stores and queries; it does not run LLMs to extract triples from text. You can build extraction on top; Topos does not require it.
- **Not a server / hosted product.** Embedded only, like SQLite/Kuzu. You hold a `HypergraphKernel` in-process.
- **Not a reasoning / entailment / belief-revision engine.** It stores `AssertionMode` (asserted/quoted/hypothesized) and `Provenance`, but it does not reason over them. No contradiction resolution, no truth maintenance, no logical entailment. Storing an `AssertionMode.Hypothesized` flag is storage; revising beliefs from it is *your* job.
- **Not multi-language by FFI.** Pure C#.

`[verified:docs=docs/SPECIFICATION.md §2.2]`

---

3. Getting started

> The examples here are adapted from the real, working code in `samples/Topos.Samples.ChatMemory/` (the M5 falsifiability sample) and its tests, stripped of the meta-framing.
> `[verified:src=samples/Topos.Samples.ChatMemory/ChatMemory.cs]`

3.1 Create a kernel and some vertices

```
using Topos.Hypergraph;

var kernel = new HypergraphKernel();

// Domain vertices — things in your domain. Allocate one, get its Handle back.
Handle alice   = kernel.CreateVertex();
Handle kyoto   = kernel.CreateVertex();
Handle nara    = kernel.CreateVertex();
Handle osaka   = kernel.CreateVertex();
```

`CreateVertex` allocates a fresh, never-reused `Handle` with `VertexStatus.Active`.
`[verified:src=src/Topos.Hypergraph/HypergraphKernel.cs:51-62]` A `Handle` is just an identity — it carries no data. Data attaches via typed properties (next step).

3.2 Define typed properties

Before you can attach data, you resolve a `PropertyKey<T>` for each typed attribute. **Resolve once, cache the result** (e.g. in a field), and reuse it for every get/set — repeated resolution is a dictionary lookup, not free, and the type `T` must be consistent for a given name.
`[verified:src=src/Topos.Hypergraph/HypergraphKernel.cs:144-152]`

```
PropertyKey<string>  name      = kernel.ResolveProperty<string>("name");
PropertyKey<float[]> embedding = kernel.ResolveProperty<float[]>("embedding");

kernel.SetProperty(name, alice, "Alice");
kernel.SetProperty(name, kyoto, "Kyoto");

if (kernel.TryGetProperty(name, alice, out var aliceName))
    Console.WriteLine(aliceName);  // "Alice"
```

A few things worth knowing up front: `[verified:src=src/Topos.Hypergraph/PropertyKey.cs:6-26]`

- `PropertyKey<T>`'s constructor is `internal` — `ResolveProperty<T>` is the only way to get one (locked M8).

- No existence check on the `Handle` in `SetProperty` — properties on a not-yet-created or dormant Handle are legal (Invariant 1: provenance edges always resolve).
- Resolving the *same name* with two different `T`s is a caller error that throws `InvalidCastException` on first pool access. Pick one `T` per name and stick to it.

3.3 Build an n-ary hyperedge

This is the heart of Topos. One utterance mentioning three entities is **one atomic relationship**, not three separate edges. Model it as one hyperedge — a vertex tagged `VertexRoles.Edge`, connected to its members by incidences: `[verified:src=samples/Topos.Samples.ChatMemory/ChatMemory.cs:71-79]`

```
// Define your domain's roles as a byte-backed enum (docs/ROLE_CONVENTIONS.md).
// [verified:docs=docs/ROLE_CONVENTIONS.md — "The decision"]
public enum TripRole : byte
{
    Speaker  = 0,  // the turn that said it
    Mention  = 1,  // an entity that was mentioned
}

// The hyperedge itself, reified as a vertex.
Handle mention = kernel.CreateVertex(VertexRoles.Edge);

// Wire up the members. Source is the edge; Member is each participant.
kernel.AddIncidence(mention, alice, (byte)TripRole.Speaker, ordinal: 0);
kernel.AddIncidence(mention, kyoto, (byte)TripRole.Mention, ordinal: 1);
kernel.AddIncidence(mention, nara,  (byte)TripRole.Mention, ordinal: 2);
kernel.AddIncidence(mention, osaka, (byte)TripRole.Mention, ordinal: 3);
```

`AddIncidence` takes the role as a raw `byte` — the kernel does not interpret it. Defining role bytes as a `byte`-backed `enum` and casting explicitly is the documented convention.
`[verified:src=src/Topos.Hypergraph/HypergraphKernel.cs:116-126]`

3.4 Query it back

One-hop: who was mentioned in this turn?

Hand-rolled (works against the kernel alone):
`[verified:src=samples/Topos.Samples.ChatMemory/ChatMemory.cs:81-85]`

```
IReadOnlyList<Handle> mentioned = kernel
    .GetVertexHyperedges(alice)              // hyperedges alice is in
    .SelectMany(edge => kernel.IncidencesFrom(edge))
    .Where(i => i.Role == (byte)TripRole.Mention)
    .Select(i => i.Member)
    .ToList();
// [kyoto, nara, osaka]
```

Equivalent, using the M9 `Knowledge` package (a one-liner once you reference it):
`[verified:src=src/Topos.Hypergraph.Knowledge/DirectedTraversal.cs:89-99]`

```
using Topos.Hypergraph.Knowledge;

IReadOnlyList<Handle> mentioned = kernel.RoleFilteredMembers(alice, (byte)TripRole.Mention);
```

Multi-hop topology: is `alice` connected to `osaka`?

Every kernel algorithm is a default method on `IHypergraphQuery`, built purely from the five required primitives — `HypergraphKernel` *is* an `IHypergraphQuery`:
`[verified:src=src/Topos.Hypergraph/IHypergraphQuery.cs:57-159]`

```
IHypergraphQuery query = kernel;

bool reachable = query.IsReachable(alice, osaka);            // true — they share the mention
hyperedge
IReadOnlyList<Handle> path = query.GetShortestPath(alice, osaka);
foreach (var v in path)
    Console.WriteLine(v);   // #0, #3  (alice → osaka, one hop across the hyperedge)
```

A subtlety: at this layer, "adjacent" means *co-incident on the same hyperedge*. The `Speaker` / `Mention` distinction is invisible to `IsReachable` — that's role-blind traversal by design ("the kernel does not judge"). The path is one hop because alice and osaka are both members of the same hyperedge.
`[verified:src=src/Topos.Hypergraph/IHypergraphQuery.cs:127-225]`

Directed: follow only `Speaker → Mention` legs

If your domain has direction, use the M9 `Knowledge` package — the kernel's own traversal is deliberately role-blind: `[verified:src=src/Topos.Hypergraph.Knowledge/DirectedTraversal.cs:19-51]`

```
// From alice (Speaker), reach everyone she mentioned — multi-hop, across hyperedges.
IReadOnlyList<Handle> directed = query.DirectedBfs(alice, (byte)TripRole.Speaker,
(byte)TripRole.Mention);
```

`DirectedBfs` follows only hyperedges where the current frontier vertex holds the `fromRole`, landing on that edge's `toRole` members. It is an extension method on `IHypergraphQuery`, so it works over a kernel, a filtered view, a union — any source.

3.5 Save and reload

To round-trip the kernel through disk, reference `Topos.Hypergraph.Persistence` and use `HypergraphSnapshot`. You save topology (vertices, incidences, allocator state) **plus an explicit, caller-specified set of typed property columns** — the snapshot machinery does not introspect arbitrary property types automatically. `[verified:src=src/Topos.Hypergraph.Persistence/HypergraphSnapshot.cs:45-88]`

```
using Topos.Hypergraph.Persistence;
using System.IO;

var columns = new IPersistedPropertyColumn[]
{
    PersistedProperty.String(name),         // the PropertyKey<string> from step 3.2
    // PersistedProperty.Single(embedding)  // float, not double — uncomment to persist
embeddings
};

using (var stream = File.Create("trip.snap"))
    HypergraphSnapshot.Save(kernel, stream, columns);

HypergraphKernel reloaded;
using (var stream = File.OpenRead("trip.snap"))
    reloaded = HypergraphSnapshot.Load(stream, columns);

// Handle identity is intact across reload — Invariant 1 preserved.
Console.WriteLine(reloaded.CountVertices() == kernel.CountVertices()); // true
```

A few non-obvious things worth knowing:
`[verified:src=src/Topos.Hypergraph.Persistence/HypergraphSnapshot.cs:25-44]`

- `Save` writes the kernel's `NextHandleIndex`; `Load` constructs the new kernel with that as its allocator's starting point, so the first vertex created post-reload gets a genuinely fresh Index — never a collision with a restored one.
- Built-in codecs cover common types (`int`, `long`, `double`, `float`, `bool`, `string`, `byte`, `DateOnly`). Anything else needs `PersistedProperty.Custom<T>` with a caller-supplied encode/decode pair. `[verified:src=src/Topos.Hypergraph.Persistence/PersistedProperty.cs:53-80]`
- This is **not** a transparent hot/cold hybrid kernel (no automatic spill under memory pressure), and **not** an LSM tree (no WAL/compaction/crash safety). It's an explicit save/load step.

---

4. API reference

> Every public type, grouped by layer/package. Each entry has a one-sentence purpose, the load-bearing public members with signatures, a when-to-use note, and a source cross-ref. Every claim is tagged `[verified:src=...]`. Internal types (not public API) are in §4.8. All types target **.NET 10**. `[verified:src=src/Topos.Hypergraph/Topos.Hypergraph.csproj]`

4.1 Kernel — `Topos.Hypergraph`

The four-primitive storage contract, the kernel class, and the query interface. Spec §3. `[verified:docs=docs/SPECIFICATION.md §3]`

`Handle`

Stable identity for a vertex. `[verified:src=src/Topos.Hypergraph/Handle.cs:17]`

```
public readonly record struct Handle(uint Index, uint Generation = 0)
{
    public static readonly Handle Invalid = new(uint.MaxValue, uint.MaxValue);
    public bool IsValid => this != Invalid;
    public override string ToString();
}
```

- `Index` — monotonic, never-reused counter. `[verified:src=src/Topos.Hypergraph/Handle.cs:6-11]`
- `Generation` — reserved for a possible future slot-compaction feature. Always `0` today. `[verified:src=src/Topos.Hypergraph/Handle.cs:11-15]`
- `Invalid` — the "no vertex" sentinel. Failure paths on `TryGetVertex` set the out `Vertex`'s Handle to this value (not C#'s `default(Handle)`, which would collide with real vertex #0). Always check the `bool`; `IsValid` is defense-in-depth. `[verified:src=src/Topos.Hypergraph/Handle.cs:19-36]`

`Vertex`

The node record. `[verified:src=src/Topos.Hypergraph/Vertex.cs:9]`

```
public readonly record struct Vertex(Handle Handle, VertexRoles Roles, VertexStatus Status)
{
    public bool IsDormant => Status == VertexStatus.Dormant;
}
```

`Roles` and `Status` are inline struct fields (read per-hop by traversal), not PropertyBag entries — the record is intentionally small. `[verified:src=src/Topos.Hypergraph/Vertex.cs:3-8]`

`Incidence`

One membership in a hyperedge. `[verified:src=src/Topos.Hypergraph/Incidence.cs:30]`

```
public readonly record struct Incidence(Handle Source, Handle Member, byte Role, int Ordinal);
```

`Source` is the edge-vertex's Handle; `Member` is a participant. `Role` is a raw byte — the kernel does not interpret it. `Ordinal` is the position. **No cell-level properties exist on `Incidence` itself** — see §5.3 for the workarounds if you need per-membership data. `[verified:src=src/Topos.Hypergraph/Incidence.cs:6-28]`

`PropertyKey<T>`

Typed identity for a property. `[verified:src=src/Topos.Hypergraph/PropertyKey.cs:16-26]`

```
public readonly record struct PropertyKey<T>
{
    internal PropertyKey(string name, int id);   // internal — locked M8
    public string Name { get; }
    public int Id { get; }
}
```

Obtain one only via `HypergraphKernel.ResolveProperty<T>(string)` (or `PropertyRegistry.Resolve<T>`) — the constructor is `internal` to prevent constructing two keys with the same `Id` but different `T`s (a footgun that throws `InvalidCastException` on first pool access).
`[verified:src=src/Topos.Hypergraph/PropertyKey.cs:6-15]`

`PropertyRegistry`

Per-process, thread-safe string→int property registry.
`[verified:src=src/Topos.Hypergraph/PropertyRegistry.cs:18-28]`

```
public sealed class PropertyRegistry
{
    public PropertyKey<T> Resolve<T>(string name);
}
```

An Id is assigned once per name and never reused. The name→Id mapping is **untyped** — resolving the same name with two different `T`s is a caller error that throws `InvalidCastException` on first pool access. In practice, most code calls `kernel.ResolveProperty<T>` rather than instantiating a `PropertyRegistry` directly.
`[verified:src=src/Topos.Hypergraph/PropertyRegistry.cs:11-16]`

`VertexRoles`

Kernel-level roles on a `Vertex` (read per-hop by traversal).
`[verified:src=src/Topos.Hypergraph/VertexRoles.cs:13-17]`

```
[Flags]
public enum VertexRoles : byte
{
    None = 0,
    Edge = 1 << 0,   // the one kernel role: "this vertex is a reified hyperedge"
}
```

Kernel-level only. Domain roles do **not** go here — they live as `Incidence.Role` bytes.
`[verified:src=src/Topos.Hypergraph/VertexRoles.cs:3-11]`

`VertexStatus`

Active/dormant tombstone flag. `[verified:src=src/Topos.Hypergraph/VertexStatus.cs:8-12]`

```
public enum VertexStatus : byte
{
    Active = 0,
    Dormant = 1,
}
```

`Dormant` is a tombstone, not deletion — Invariant 1.
`[verified:src=src/Topos.Hypergraph/VertexStatus.cs:3-7]`

`HandleAllocator`

Lock-free monotonic Handle allocator. `[verified:src=src/Topos.Hypergraph/HandleAllocator.cs:8-24]`

```
public sealed class HandleAllocator
{
    public HandleAllocator(uint startingIndex = 0);
    public Handle Next();
    public uint NextIndex { get; }
}
```

Safe for concurrent callers (`Interlocked.Increment`). Never reuses an Index — Invariant 1 depends on this. `startingIndex` exists for snapshot reload (M4). Most code never touches this class directly — `HypergraphKernel` owns its own instance. `[verified:src=src/Topos.Hypergraph/HandleAllocator.cs:4-22]`

`HypergraphKernel`

The kernel: storage + writes. Implements `IHypergraphQuery` .
`[verified:src=src/Topos.Hypergraph/HypergraphKernel.cs:32]`

```
public sealed class HypergraphKernel : IHypergraphQuery
{
    public HypergraphKernel();
    public HypergraphKernel(uint startingHandleIndex);   // M4: resume after reload
    public uint NextHandleIndex { get; }                 // M4: snapshot-side counterpart

    // ── Vertices ──
    public Handle CreateVertex(VertexRoles roles = VertexRoles.None);
    public bool TryGetVertex(Handle handle, out Vertex vertex);
    public void RestoreVertex(Handle handle, VertexRoles roles, VertexStatus status); // M4,
snapshot only
    public void SetDormant(Handle handle);
    public void Reactivate(Handle handle);

    // ── Incidences ──
    public Incidence AddIncidence(Handle source, Handle member, byte role, int ordinal);
    public ImmutableArray<Incidence> IncidencesFrom(Handle source);
    public ImmutableArray<Incidence> IncidencesOf(Handle member);
    public IEnumerable<Incidence> AllIncidences();   // added for M4 snapshot consumer

    // ── Properties ──
    public PropertyKey<T> ResolveProperty<T>(string name);
    public void SetProperty<T>(PropertyKey<T> key, Handle handle, T value);
    public bool TryGetProperty<T>(PropertyKey<T> key, Handle handle, out T value);
    public bool RemoveProperty<T>(PropertyKey<T> key, Handle handle);
    public ImmutableArray<(Handle Handle, T Value)> EnumerateProperty<T>(PropertyKey<T> key);

    // ── IHypergraphQuery primitives (see below) ──
    public int CountVertices();
    public IReadOnlyList<Handle> VertexHandles();
    public IReadOnlyList<Handle> GetVertexHyperedges(Handle vertex);
    public IReadOnlyList<Incidence> GetHyperedgeVertices(Handle hyperedge);
}
```

Key operational facts: `[verified:src=src/Topos.Hypergraph/HypergraphKernel.cs:43–172]`

- **Concurrency:** Single-Writer/Multi-Reader. Handle allocation is lock-free; everything else is a sparse-set behind its own `ReaderWriterLockSlim`. `[verified:src=src/Topos.Hypergraph/HypergraphKernel.cs:6–31]`
- **No existence checks** on `AddIncidence` / `SetProperty` — provenance edges always resolve, even to dormant targets (Invariant 1). `[verified:src=src/Topos.Hypergraph/HypergraphKernel.cs:116–126]`

`[verified:src=src/Topos.Hypergraph/HypergraphKernel.cs:154-156]`

- `RestoreVertex` is the **only sanctioned Invariant-1 bypass** — inserts a vertex at an already-allocated Handle, bypassing the allocator. Snapshot reload only; its only caller is `HypergraphSnapshot.Load`. `[verified:src=src/Topos.Hypergraph/HypergraphKernel.cs:71-81]`

`IHypergraphQuery`

The query surface: 5 required primitives + ~40 default-implemented algorithms. Spec §6 M1. `[verified:src=src/Topos.Hypergraph/IHypergraphQuery.cs:57]`

```
public interface IHypergraphQuery
{
    // ── Required primitives ──
    int CountVertices();
    IReadOnlyList<Handle> VertexHandles();
    bool TryGetVertex(Handle handle, out Vertex vertex);
    IReadOnlyList<Handle> GetVertexHyperedges(Handle vertex);
    IReadOnlyList<Incidence> GetHyperedgeVertices(Handle hyperedge);

    // ── Default-implemented derivations ──
    bool IsEmpty();
    bool ContainsVertex(Handle handle);
    Vertex GetVertex(Handle handle);                        // throwing counterpart of
TryGetVertex
    IReadOnlyList<Handle> HyperedgeHandles();               // O(V) scan for VertexRoles.Edge
    int CountHyperedges();

    // ── Default-implemented algorithms ──
    IEnumerable<Handle> GetBfs(Handle start);
    IEnumerable<Handle> GetDfs(Handle start);
    bool IsReachable(Handle from, Handle to);
    int? GetShortestPathLength(Handle from, Handle to);     // null if unreachable
    IReadOnlyList<Handle> GetShortestPath(Handle from, Handle to);
    bool HasCycle();
    IReadOnlyDictionary<Handle, IReadOnlyList<Handle>> GetTransitiveClosure();
    IReadOnlyList<IReadOnlyList<Handle>> GetConnectedComponents();
}
```

Every algorithm here is role-blind and co-membership-symmetric ("the kernel does not judge"). A hop is one vertex→vertex step across one hyperedge; role direction is invisible. For role-aware / directed traversal, use `Topos.Hypergraph.Knowledge` (§4.7). `[verified:src=src/Topos.Hypergraph/IHypergraphQuery.cs:44-56]`

Two warnings worth reading in the source rather than paraphrased:

- **`HasCycle()` returns `true` almost always on any real n-ary hypergraph.** Three co-members are pairwise "adjacent," so any hyperedge with 3+ members is trivially cyclic at this layer. `[verified:src=src/Topos.Hypergraph/IHypergraphQuery.cs:277-306]`
- **`GetConnectedComponents()` is deliberately not called "SCC."** At the topology layer the adjacency is symmetric, so strongly/weakly connected coincide. GDS-verified against `gds.wcc`. `[verified:src=src/Topos.Hypergraph/IHypergraphQuery.cs:358-410]`

GDS-parity coverage: BFS, DFS, and `GetConnectedComponents` are verified against `gds.bfs` / `gds.dfs` / `gds.wcc` in `tests/Topos.Tests.GdsOracle`. `GetShortestPathLength` is verified with a documented bipartite-vs-logical hop correction. Cycle detection and transitive closure are unit-tested only. `[verified:src=src/Topos.Hypergraph/IHypergraphQuery.cs:36-42]`

4.2 Reification & semantics — `Topos.Hypergraph`

Two public types that model epistemic status and provenance. **Neither is a reserved `Vertex` field** — both are stored via `PropertyKey<T>` pools like any typed attribute, which is itself a small validation that M0's property design was already general enough for M2/M5.

> There is **no dedicated reification method** on the kernel. Reification is a usage pattern: `CreateVertex(VertexRoles.Edge)` + `AddIncidence`. Nested reification (an edge that participates in another edge) is just an `Incidence` whose `Member` is an edge-vertex. `[verified:docs=docs/SPECIFICATION.md §7 pattern 12]`

`AssertionMode`

Epistemic status of a reified relationship. `[verified:src=src/Topos.Hypergraph/AssertionMode.cs:20-30]`

```
public enum AssertionMode : byte
{
    Asserted = 0,      // committed as true
    Quoted = 1,        // referenced without commitment (RDF 1.2 unasserted triple term)
    Hypothesized = 2,  // candidate belief, not yet confirmed/rejected
}
```

Not a reserved field — nothing in core traversal reads it per-hop. Resolve a property key for it: `kernel.ResolveProperty<AssertionMode>("mode")`. A vertex with no mode property set means "no mode recorded." `[verified:src=src/Topos.Hypergraph/AssertionMode.cs:3-19]`

`Provenance`

Where a fact came from. `[verified:src=src/Topos.Hypergraph/Provenance.cs:17]`

```
public readonly record struct Provenance(string Source, DateTimeOffset RecordedAt);
```

"First-class" here means a named designed type with clear semantics, not a new storage mechanism. **For structural provenance** — which other in-graph facts a fact was derived from — nested reification is the actual mechanism: link a derived edge to its source edges via `Incidence`. This record is for the **leaf** case: provenance that terminates outside the graph (a document, a user, an external system).
`[verified:src=src/Topos.Hypergraph/Provenance.cs:4-16]`

4.3 Views & set algebra — `Topos.Hypergraph`

Composable read-only views over `IHypergraphQuery`. Spec §6 M3.

`HypergraphViews`

Static factory for composable views. All methods return `IHypergraphQuery` and are O(1) to construct.
`[verified:src=src/Topos.Hypergraph/HypergraphViews.cs:37-57]`

```
public static class HypergraphViews
{
    public static IHypergraphQuery Subgraph(IHypergraphQuery source, Func<Handle, bool>
predicate);
    public static IHypergraphQuery Mask(IHypergraphQuery source, Func<Handle, bool> predicate);
// same as Subgraph
    public static IHypergraphQuery Union(IHypergraphQuery a, IHypergraphQuery b);
    public static IHypergraphQuery Intersect(IHypergraphQuery a, IHypergraphQuery b);
    public static IHypergraphQuery Difference(IHypergraphQuery a, IHypergraphQuery b);
}
```

Notes: `[verified:src=src/Topos.Hypergraph/HypergraphViews.cs:3-36]`

- **No `Unmodifiable` view exists, deliberately.** `IHypergraphQuery` has no write members, so typing a kernel as `IHypergraphQuery` already gives an unmodifiable view at compile time, for free.
- **Set algebra doubles as in-kernel version-diff.** Because `Handle.Index` is monotonic and never reused, `Subgraph(kernel, h => h.Index < threshold)` is a genuine "state as of an earlier point in this kernel's history." `Difference(later, earlier)` gives "what's been added since." See §5.4.

`FilteredView`

Read-only view restricting a source to vertices passing a predicate.
`[verified:src=src/Topos.Hypergraph/FilteredView.cs:24-42]`

```
public sealed class FilteredView(IHypergraphQuery source, Func<Handle, bool> predicate) :
IHypergraphQuery;
```

`Subgraph` and `Mask` both construct one of these. A hyperedge is included only when the predicate accepts the edge-vertex itself; a member is reported only when the predicate also accepts that member (JGraphT `AsSubgraph` convention generalized to N-ary — out-of-view members are silently dropped, not errors). Every

`IHypergraphQuery` algorithm works over a `FilteredView` unchanged. `[verified:src=src/Topos.Hypergraph/FilteredView.cs:11–23]`

`UnionView`

Read-only view presenting the union of two sources. `[verified:src=src/Topos.Hypergraph/UnionView.cs:24–42]`

```
public sealed class UnionView(IHypergraphQuery a, IHypergraphQuery b) : IHypergraphQuery;
```

Conflict rule: `a` wins if the same Handle resolves differently in both. **Only meaningful when both sources share a Handle-identity space** — two views from the same kernel qualify; two independently-constructed kernels do not. `[verified:src=src/Topos.Hypergraph/UnionView.cs:8–23]`

4.4 Embeddings & learnable edges — `Topos.Hypergraph`

Derived structures over `PropertyKey<T>` data. **None of these adds kernel storage** — they read from the kernel's existing typed properties via `EnumerateProperty`, so each is swappable without touching `HypergraphKernel`. Spec §6 M5.

`VectorIndex`

k-nearest-neighbor search over `PropertyKey<float[]>` embeddings. `[verified:src=src/Topos.Hypergraph/VectorIndex.cs:20–56]`

```
public sealed class VectorIndex(HypergraphKernel kernel, PropertyKey<float[]> embeddingKey)
{
    public IReadOnlyList<(Handle Handle, float Distance)> NearestNeighbors(ReadOnlySpan<float>
query, int k);
}
```

Brute-force, not approximate — the name says "VectorIndex," not "ApproximateNearestNeighborIndex," to avoid overclaiming. Squared Euclidean distance; throws on `k <= 0` or embedding-dimension mismatch (no padding/truncation). `[verified:src=src/Topos.Hypergraph/VectorIndex.cs:3–19]`

`LearnableEdge`

Sigmoid edge weight, reinforced by gradient ascent on reward. `[verified:src=src/Topos.Hypergraph/LearnableEdge.cs:15–56]`

```
public readonly record struct LearnableEdge(float[] Theta)
{
    public float Evaluate(ReadOnlySpan<float> features);                // theta[0] is bias
    public LearnableEdge Reinforce(ReadOnlySpan<float> features, float reward, float
learningRate);
    public static LearnableEdge CreateUninitialized(int featureCount);   // all-zero → Evaluate
returns 0.5
}
```

Generalizes RLB's `ThetaParameters` / `ReinforceTheta` without RLB's fixed feature layout. Immutable value type: `Reinforce` returns a *new* instance; the caller `SetProperty`s it back over the old one. `[verified:src=src/Topos.Hypergraph/LearnableEdge.cs:3-14]`

`EdgeStatistics`

Per-membership statistics carried on an edge. `[verified:src=src/Topos.Hypergraph/EdgeStatistics.cs:14-29]`

```
public readonly record struct EdgeStatistics(int TransitionCount, double SuccessRate, double
Confidence)
{
    public static readonly EdgeStatistics Initial = new(0, 1.0, 0.5);
    public EdgeStatistics Observe(bool succeeded, double smoothing = 0.1);  // EMA update
}
```

Generalizes RLB's `TransitionCount` / `SuccessRate` / `Confidence`. The EMA `Observe` rule is a sensible default, not a mandated one. `[verified:src=src/Topos.Hypergraph/EdgeStatistics.cs:3-13]`

4.5 Analytics — `Topos.Hypergraph`

M6 algorithms over topology-only bipartite adjacency. Spec §6 M6.

`SWalk`

s-walk / s-distance over hyperedges. The one genuinely hypergraph-specific algorithm here. `[verified:src=src/Topos.Hypergraph/SWalk.cs:15-105]`

```
public static class SWalk
{
    public static IEnumerable<Handle> Reachable(IHypergraphQuery graph, Handle start, int s);
    public static int? Distance(IHypergraphQuery graph, Handle from, Handle to, int s);
}
```

Two hyperedges are **s-adjacent** when they share ≥ `s` common members. **Deliberately not GDS-verified** — GDS operates on the binary projection and has no notion of "share ≥ s members"; Topos's answer is the novel claim here. Both methods throw `ArgumentOutOfRangeException` eagerly (not on enumeration) if `s < 1`. `[verified:src=src/Topos.Hypergraph/SWalk.cs:3-14]` `[verified:src=src/Topos.Hypergraph/SWalk.cs:17-31]`

`LabelPropagation`

Community detection by label propagation. GDS-verified (`gds.labelPropagation`). `[verified:src=src/Topos.Hypergraph/LabelPropagation.cs:17-65]`

```
public static class LabelPropagation
{
    public static IReadOnlyDictionary<Handle, int> DetectCommunities(IHypergraphQuery graph, int
maxIterations = 100);
}
```

Returns a vertex→community-id map. Ties broken by lowest label; fixed seed for deterministic output; isolated vertices keep their own label. **Chosen over Louvain deliberately** — Louvain is substantially more complex to get right, and Label Propagation gives real, GDS-verified community detection today. `[verified:src=src/Topos.Hypergraph/LabelPropagation.cs:3-15]`

`TriangleCount`

Triangle counting over bipartite adjacency. GDS-verified (`gds.triangleCount`). `[verified:src=src/Topos.Hypergraph/TriangleCount.cs:24-50]`

```
public static class TriangleCount
{
    public static long Count(IHypergraphQuery graph);
}
```

Non-obvious consequence of the bipartite reification: an N-member hyperedge and its members form a complete graph on N+1 vertices, giving `C(N+1, 3)` triangles from one hyperedge. A plain 2-member edge already yields 1 triangle (`C(3,3) = 1`); a 3-member edge yields 4. `[verified:src=src/Topos.Hypergraph/TriangleCount.cs:13-23]`

`Modularity`

Newman's modularity Q for a given partition. **A scorer, not a detector** — pass it `LabelPropagation.DetectCommunities`'s output. `[verified:src=src/Topos.Hypergraph/Modularity.cs:12-72]`

```
public static class Modularity
{
    public static double Compute(IHypergraphQuery graph, IReadOnlyDictionary<Handle, int>
communities);
}
```

Returns `0.0` for an edgeless graph. **Asymmetry to know:** vertices missing from `communities` are excluded from the internal-edges numerator but grouped into a synthetic community (key `-1`) for the degree-sum-of-squares penalty — so omitting vertices can only push Q down, never up. Pass a partition covering every vertex for a meaningful score. `[verified:src=src/Topos.Hypergraph/Modularity.cs:15-28]`

`Centrality`

Standard centrality measures (M11 phase 1). GDS-verified (`gds.degree` / `gds.closeness` / `gds.betweenness`). `[verified:src=src/Topos.Hypergraph/Centrality.cs:15-132]`

```
public static class Centrality
{
    public static IReadOnlyDictionary<Handle, int> Degree(IHypergraphQuery graph);
    public static IReadOnlyDictionary<Handle, double> Closeness(IHypergraphQuery graph);
    public static IReadOnlyDictionary<Handle, double> Betweenness(IHypergraphQuery graph);
}
```

Shares `Modularity` / `TriangleCount` / `LabelPropagation`'s bipartite (clique-expansion) adjacency, not M1's member-only `GetBfs` adjacency. **`Degree`** — size of each vertex's distinct neighbor set. **`Closeness`** — `k / Σ(distance to each of the k reachable others)`, GDS's actual default formula (`useWassermanFaustFormula = false`) — each vertex scores against only its own reachable set, so disconnected components each get an internally-consistent score. **`Betweenness`** — Brandes' algorithm (one BFS per source), halved at the end (the standard undirected-graph correction). All three now confirmed exact-value parity against a live Neo4j+GDS instance (2026-08-03).

`PageRank`

Standard PageRank (M11 phase 1). GDS-verified (`gds.pageRank`). `[verified:src=src/Topos.Hypergraph/PageRank.cs:13-56]`

```
public static class PageRank
{
    public static IReadOnlyDictionary<Handle, double> Compute(
        IHypergraphQuery graph, double damping = 0.85, int maxIterations = 100, double tolerance
= 1e-6);
}
```

Power iteration over the same adjacency `Centrality` uses — symmetric co-membership, kernel-level. Uniform dangling-mass redistribution; damping `0.85` matches `gds.pageRank`'s own default. Converges to a distribution summing to `1.0` across every vertex. **Note:** `gds.pageRank`'s own default output is *not* normalized to sum to 1 (its base term is `(1-d)`, not `(1-d)/N`) — comparing against GDS means L1-normalizing GDS's raw scores first; this is a genuine convention difference, not a bug in either implementation.

4.6 Persistence — `Topos.Hypergraph.Persistence`

Save/load a kernel's topology + caller-specified property columns to/from a `Stream`. Spec §6 M4.

`HypergraphSnapshot`

Static save/load entry points. `[verified:src=src/Topos.Hypergraph.Persistence/HypergraphSnapshot.cs:45-139]`

```
public static class HypergraphSnapshot
{
    public static void Save(HypergraphKernel kernel, Stream stream,
IReadOnlyList<IPersistedPropertyColumn>? properties = null);
    public static HypergraphKernel Load(Stream stream, IReadOnlyList<IPersistedPropertyColumn>?
properties = null);
}
```

Versioned binary format (magic `0x53485054` "TPHS", format version 1). Columnar for properties. **Invariant 1 preserved across reload:** `Save` writes `NextHandleIndex`; `Load` constructs the new kernel with that as the allocator's start, so the first post-reload vertex gets a fresh Index, never a collision.
`[verified:src=src/Topos.Hypergraph.Persistence/HypergraphSnapshot.cs:16-23]`

Scope, plainly stated: this is *not* a transparent hot/cold hybrid kernel and *not* an LSM tree (no WAL/compaction/crash safety beyond a completed write). The transparent tiered version is real follow-on work.
`[verified:src=src/Topos.Hypergraph.Persistence/HypergraphSnapshot.cs:25-44]`

`IPersistedPropertyColumn`

Non-generic handle for a heterogeneous list of typed property columns.
`[verified:src=src/Topos.Hypergraph.Persistence/PersistedProperty.cs:9-14]`

```
public interface IPersistedPropertyColumn
{
    string Name { get; }
    void WriteTo(HypergraphKernel kernel, BinaryWriter writer);
    void ReadFrom(HypergraphKernel kernel, BinaryReader reader);
}
```

Lets `Save` / `Load` take one flat `IReadOnlyList` mixing columns of different `T`s. The generic implementing class `PersistedPropertyColumn<T>` is `internal`.

`PersistedProperty`

Factory for column codecs — common types plus a custom escape hatch.
`[verified:src=src/Topos.Hypergraph.Persistence/PersistedProperty.cs:51-81]`

```
public static class PersistedProperty
{
    public static IPersistedPropertyColumn Int32(PropertyKey<int> key);
    public static IPersistedPropertyColumn Int64(PropertyKey<long> key);
    public static IPersistedPropertyColumn Double(PropertyKey<double> key);
    public static IPersistedPropertyColumn Single(PropertyKey<float> key);
    public static IPersistedPropertyColumn Boolean(PropertyKey<bool> key);
    public static IPersistedPropertyColumn String(PropertyKey<string> key);
    public static IPersistedPropertyColumn Byte(PropertyKey<byte> key);
    public static IPersistedPropertyColumn DateOnly(PropertyKey<DateOnly> key);   // day-number
encoded
    public static IPersistedPropertyColumn Custom<T>(PropertyKey<T> key, Action<BinaryWriter, T>
write, Func<BinaryReader, T> read);
}
```

Deliberately not fully-generic-automatic — the caller states up front which properties to persist and how.
`[verified:src=src/Topos.Hypergraph.Persistence/PersistedProperty.cs:45-50]`

`LruCache<TKey, TValue>`

Classic O(1) LRU cache — the tested hot-tier building block for future tiered storage.
`[verified:src=src/Topos.Hypergraph.Persistence/LruCache.cs:13]`

```
public sealed class LruCache<TKey, TValue> where TKey : notnull
{
    public LruCache(int capacity);
    public int Capacity { get; }
    public int Count { get; }
    public bool TryGet(TKey key, out TValue value);
    public bool ContainsKey(TKey key);
    public (TKey Key, TValue Value)? Set(TKey key, TValue value);  // returns evicted pair, if
any
    public bool Remove(TKey key);
}
```

Dictionary + intrusive doubly-linked list. Not thread-safe on its own. `Set` returns the evicted pair if the insertion caused an eviction — the caller's cue to flush to the cold tier.
`[verified:src=src/Topos.Hypergraph.Persistence/LruCache.cs:3-12]`

4.7 Knowledge — `Topos.Hypergraph.Knowledge`

Layer-1 role-aware directed traversal. Spec §6 M9. A separate package (own assembly); pure consumer of `IHypergraphQuery` — **no kernel changes.** Generalizes a pattern three independent consumers (ChatMemory, NexusVerifier, Rich-Learning-Base's `ToposGraphProjection`) each hand-rolled.
`[verified:src=src/Topos.Hypergraph.Knowledge/Topos.Hypergraph.Knowledge.csproj]`

`DirectedTraversal`

Role-aware directed traversal over `IHypergraphQuery`.
`[verified:src=src/Topos.Hypergraph.Knowledge/DirectedTraversal.cs:17-113]`

```
public static class DirectedTraversal
{
    // BFS following only hyperedges where the frontier vertex holds fromRole, landing on toRole
members.
    public static IReadOnlyList<Handle> DirectedBfs(this IHypergraphQuery graph, Handle start,
byte fromRole, byte toRole);

    // One shortest directed path following only fromRole→toRole legs. Empty if unreachable;
[from] if from==to.
    public static IReadOnlyList<Handle> DirectedShortestPath(this IHypergraphQuery graph, Handle
from, Handle to, byte fromRole, byte toRole);

    // One-hop: members of vertex's hyperedges holding the given role.
    public static IReadOnlyList<Handle> RoleFilteredMembers(this IHypergraphQuery graph, Handle
vertex, byte role);

    // M11 phase 1: strongly-connected components over the fromRole→toRole directed adjacency.
    // Every vertex gets a component, including singletons for vertices that never hold
fromRole/toRole.
    public static IReadOnlyList<IReadOnlyList<Handle>> DirectedScc(this IHypergraphQuery graph,
byte fromRole, byte toRole);
}
```

This is where "the kernel does not judge" gets its judgment: given a directed reading of hyperedge roles, walk only along that direction. All four are extension methods on `IHypergraphQuery`, so they work over a kernel, a `FilteredView`, a `UnionView` — any source.
`[verified:src=src/Topos.Hypergraph.Knowledge/DirectedTraversal.cs:3-16]`

`DirectedScc` is the directed counterpart to the kernel's `GetConnectedComponents` (WCC-equivalent) — iterative Tarjan, GDS-verifiable via `gds.scc` over a role-projected directed graph. It's the tested replacement for the class of hand-rolled directed-cycle guard NexusVerifier's finding #4 describes (don't reach for `HasCycle` here — it's role-blind and trivially `true` on any real n-ary hypergraph).

`RoleExtensions`

Turns `docs/ROLE_CONVENTIONS.md`'s byte-backed-enum pattern into real code — typed-role overloads of the above, plus `AddIncidence<TRole>`. `[verified:src=src/Topos.Hypergraph.Knowledge/RoleExtensions.cs:15-54]`

```
public static class RoleExtensions
{
    public static Incidence AddIncidence<TRole>(this HypergraphKernel kernel, Handle source,
Handle member, TRole role, int ordinal)
        where TRole : unmanaged, Enum;

    public static IReadOnlyList<Handle> DirectedBfs<TRole>(this IHypergraphQuery graph, Handle
start, TRole fromRole, TRole toRole)
        where TRole : unmanaged, Enum;

    public static IReadOnlyList<Handle> DirectedShortestPath<TRole>(this IHypergraphQuery graph,
Handle from, Handle to, TRole fromRole, TRole toRole)
        where TRole : unmanaged, Enum;

    public static IReadOnlyList<Handle> RoleFilteredMembers<TRole>(this IHypergraphQuery graph,
Handle vertex, TRole role)
        where TRole : unmanaged, Enum;

    public static IReadOnlyList<IReadOnlyList<Handle>> DirectedScc<TRole>(this IHypergraphQuery
graph, TRole fromRole, TRole toRole)
        where TRole : unmanaged, Enum;
}
```

`TRole` must be a **byte-backed** enum (`enum Foo : byte`) — a wider underlying type (e.g. `int`) throws `ArgumentException`. `[verified:src=src/Topos.Hypergraph.Knowledge/RoleExtensions.cs:37-53]`

Usage: `[verified:src=tests/Topos.Hypergraph.Knowledge.Tests/DirectedTraversalTests.cs:38-46]`

```
public enum ChainerRole : byte { Anchor = 0, Condition = 1, Target = 2 }

IHypergraphQuery query = kernel;
var reachable = query.DirectedBfs(start, ChainerRole.Anchor, ChainerRole.Target);
kernel.AddIncidence(edge, target, ChainerRole.Target, ordinal: 2);
```

4.8 Internal types — not public API

These exist behind `InternalsVisibleTo` (granted to `Topos.Hypergraph.Tests` and `Topos.Hypergraph.Benchmarks` in `Topos.Hypergraph.csproj`) for direct test/benchmark access. They are **not** part of the public surface and may change without notice.
`[verified:src=src/Topos.Hypergraph/Topos.Hypergraph.csproj — InternalsVisibleTo entries]`

Type	Source	What it is
`SparseSet<T>`	`src/Topos.Hypergraph/SparseSet.cs`	EnTT-style columnar pool backing every property/vertex store. O(1) add/remove/lookup; the kernel's public surface is `IHypergraphQuery` / `HypergraphKernel`, not this. `[verified:src=src/Topos.Hypergraph/SparseSet.cs:18–24]`
`PropertyPool<T>`	`src/Topos.Hypergraph/PropertyPool.cs`	A `SparseSet<T>` behind its own `ReaderWriterLockSlim` — the per-pool-lock of spec §3.4.
`IncidenceIndex`	`src/Topos.Hypergraph/IncidenceIndex.cs`	Key→many-Incidences index backing both kernel directions; replaced an O(N²) copy-on-write design.
`BipartiteAdjacency`	`src/Topos.Hypergraph/BipartiteAdjacency.cs`	Shared neighbor-gathering for M6 analytics. `[verified:src=src/Topos.Hypergraph/BipartiteAdjacency.cs:13]`
`PersistedPropertyColumn<T>`	`src/Topos.Hypergraph.Persistence/PersistedProperty.cs:16`	Generic implementing class of `IPersistedPropertyColumn`. `[verified:src=src/Topos.Hypergraph.Persistence/PersistedProperty.cs:16–43]`

If you find yourself wanting one of these from a consumer, that's a signal worth reporting — either your use case is genuinely novel (and the public surface should grow), or there's a public-API way to do what you want that you haven't found. File it as a finding rather than reaching for `InternalsVisibleTo`.

---

5. Usage patterns

> Every pattern here is built **purely from the public API** (`Topos.Hypergraph` , optionally `Topos.Hypergraph.Knowledge` , optionally `Topos.Hypergraph.Persistence`) — no internal access, no kernel changes. That's not a constraint of this doc; it's the falsifiability standard the kernel itself holds to (see the M5 sample). `[verified:src=samples/Topos.Samples.ChatMemory/ChatMemory.cs:5-18]`

5.1 N-ary facts as one hyperedge

When to use: whenever a single event jointly involves N participants and the *togetherness* is load-bearing — one utterance mentioning several entities, one decision jointly gated by several conditions, one transaction touching several accounts. **The whole reason Topos exists.**

The thesis (spec §1): a binary graph can encode the *topology* of an n-ary event as a star, but it loses the *atomicity* — joint credit assignment, joint statistics, and the "all-conditions-present" gate have no faithful home on per-leg edges. One hyperedge preserves all three. `[verified:docs=docs/SPECIFICATION.md §1.2]`

```
using Topos.Hypergraph;

var kernel = new HypergraphKernel();

// A turn that mentioned three entities — one atomic relationship, not three edges.
public enum MentionRole : byte { Speaker = 0, Entity = 1 }

Handle turn  = kernel.CreateVertex();
Handle kyoto = kernel.CreateVertex();
Handle nara  = kernel.CreateVertex();
Handle osaka = kernel.CreateVertex();

Handle mention = kernel.CreateVertex(VertexRoles.Edge);
kernel.AddIncidence(mention, turn,  (byte)MentionRole.Speaker, ordinal: 0);
kernel.AddIncidence(mention, kyoto, (byte)MentionRole.Entity,  ordinal: 1);
kernel.AddIncidence(mention, nara,  (byte)MentionRole.Entity,  ordinal: 2);
kernel.AddIncidence(mention, osaka, (byte)MentionRole.Entity,  ordinal: 3);
```

One hyperedge, four members — and "these three were mentioned *together*, in this one turn" is stored structurally, not reconstructed from three separate edges. This is the exact shape of `samples/Topos.Samples.ChatMemory.RecordMention` .
`[verified:src=samples/Topos.Samples.ChatMemory/ChatMemory.cs:66-79]`

Variation — directed n-ary (Anchor fires toward Target, gated by Conditions): this is RLB's `HyperEdge` shape, generalized. Same primitives, different role bytes. `[verified:docs=docs/SPECIFICATION.md §1.1]`

```
public enum ChainerRole : byte { Anchor = 0, Condition = 1, Target = 2 }

Handle anchor    = kernel.CreateVertex();
Handle condition = kernel.CreateVertex();
Handle target    = kernel.CreateVertex();

Handle decision = kernel.CreateVertex(VertexRoles.Edge);
kernel.AddIncidence(decision, anchor,    (byte)ChainerRole.Anchor,    ordinal: 0);
kernel.AddIncidence(decision, condition, (byte)ChainerRole.Condition, ordinal: 1);
kernel.AddIncidence(decision, target,    (byte)ChainerRole.Target,    ordinal: 2);
```

5.2 Reification: edges as vertices

When to use: when you need to say something *about* a relationship — attach properties to it, link it into another relationship, or record its epistemic status.

Reification is already built into the kernel: a hyperedge *is* a vertex tagged `VertexRoles.Edge`. So you can attach properties to it, link it into other edges, and treat it as first-class — no special API.

5.2a — Epistemic mode (asserted / quoted / hypothesized)

```
PropertyKey<AssertionMode> mode = kernel.ResolveProperty<AssertionMode>("mode");

// A fact extracted from a turn, marked as a candidate belief (not yet confirmed).
Handle fact = kernel.CreateVertex(VertexRoles.Edge);
kernel.SetProperty(mode, fact, AssertionMode.Hypothesized);

AssertionMode? m = kernel.TryGetProperty(mode, fact, out var v) ? v : null;
```

`AssertionMode` is a plain typed property — not a reserved Vertex field — so a missing value means "no mode recorded," not "defaulted to Asserted." Interpretation is a layer-1 concern: Topos stores the flag, your code decides what to do with it. `[verified:src=samples/Topos.Samples.ChatMemory/ChatMemory.cs:88-99]`

5.2b — Nested reification (structural provenance)

An edge can be a *member* of another edge — because an edge is just a vertex. This is how you record "fact F was derived from facts G and H" as structural provenance, not just a string label.

```
// Two source facts.
Handle factG = kernel.CreateVertex(VertexRoles.Edge);
Handle factH = kernel.CreateVertex(VertexRoles.Edge);

// A derived fact, linked back to its sources via Incidence.
public enum DerivationRole : byte { Derived = 0, Source = 1 }

Handle factF = kernel.CreateVertex(VertexRoles.Edge);
kernel.AddIncidence(factF, factG, (byte)DerivationRole.Source,  ordinal: 0);  // factG is a
MEMBER here
kernel.AddIncidence(factF, factH, (byte)DerivationRole.Source,  ordinal: 1);
```

This nests to arbitrary depth — `ReificationTests.DepthN_ChainOfNestedEdges_EveryLevelRoundTrips` exercises a depth-4 chain, and every level resolves correctly via `IncidencesFrom` / `IncidencesOf`. The two incidence indexes are independent, so a vertex being an edge *and* a member of another edge don't interfere. `[verified:src=tests/Topos.Hypergraph.Tests/ReificationTests.cs:28-57]`

> **Provenance has two flavors — use the right one.** `Provenance(Source, RecordedAt)` (a typed property) is for *leaf* provenance: where a fact came from *outside* the graph. Nested reification is for *structural* provenance: which other *in-graph* facts a fact was derived from. Use both together.
> `[verified:src=src/Topos.Hypergraph/Provenance.cs:10-16]`

5.3 Per-membership ("cell") data without a kernel primitive

When to use: when you need data that varies per *membership* — per (source, member) pair — rather than per vertex or per edge. Example: a confidence score that's different for each leg of a multi-leg decision.

There is no `SetProperty(key, incidence, value)` on the kernel, by decision (M8). The two real consumers that wanted per-cell data both got by without it. `[verified:src=src/Topos.Hypergraph/Incidence.cs:16-28]` Two workarounds:

5.3a — Reify the membership as its own edge-vertex

If the per-leg data is rich enough to deserve first-class status, make each membership a vertex:

```
PropertyKey<double> legConfidence = kernel.ResolveProperty<double>("legConfidence");

// Reify the (decision, condition) membership itself as a vertex, attach data to it.
Handle leg = kernel.CreateVertex(VertexRoles.Edge);
kernel.AddIncidence(leg, decision,  (byte)ChainerRole.Anchor,    ordinal: 0);
kernel.AddIncidence(leg, condition, (byte)ChainerRole.Condition, ordinal: 1);
kernel.SetProperty(legConfidence, leg, 0.83);
```

This is "edges-as-vertices" applied one more level (Pattern 5.2b). Each membership is now a first-class vertex with ordinary per-Handle properties.

5.3b — Keep a side index in your own code

If the per-leg data is just a lookup table, keep it outside the kernel:

```
// Keyed on the (Source, Member, Ordinal) triple — whatever identifies the cell.
var legConfidence = new Dictionary<(Handle Source, Handle Member, int Ordinal), double>();

legConfidence[(decision, condition, 1)] = 0.83;
```

Both are the patterns real consumers already use; there's no kernel-level shortcut. Pick (a) when the data deserves to participate in traversal/analytics (it becomes a real vertex), (b) when it's pure side-state that the graph never needs to walk over.

5.4 Composable views

When to use: to query a *slice* of a kernel without copying it, or to combine two kernels/views set-algebraically. All views implement `IHypergraphQuery`, so every algorithm works over them unchanged.

5.4a — Subgraph / mask

```
using static Topos.Hypergraph.HypergraphViews;

// A live view of only the active (non-dormant) vertices — re-evaluated on every call.
IHypergraphQuery activeView = Mask(kernel, h =>
    kernel.TryGetVertex(h, out var v) && v.Status == VertexStatus.Active);

// BFS over the masked view — sees only active vertices.
foreach (var v in activeView.GetBfs(start)) { /* ... */ }
```

`Subgraph` and `Mask` are the same mechanism (a `FilteredView`) under two names. A member outside the view is silently dropped from an edge's reported membership, not an error.
`[verified:src=src/Topos.Hypergraph/FilteredView.cs:11-23]`

5.4b — Union / intersect / difference

```
IHypergraphQuery both     = Union(viewA, viewB);
IHypergraphQuery shared   = Intersect(viewA, viewB);
IHypergraphQuery onlyA    = Difference(viewA, viewB);
```

Union conflict rule: `a` **wins** if a Handle resolves differently in both. **Only meaningful when both sources share a Handle-identity space** — two views from the same kernel qualify; two independently-constructed kernels do not. `[verified:src=src/Topos.Hypergraph/UnionView.cs:8-23]`

5.4c — Version-diff via monotonic Handle.Index (in-kernel, no persistence)

A useful trick: because `Handle.Index` is monotonic and never reused, a Handle-Index threshold is a genuine temporal cut of one kernel's history. `[verified:src=src/Topos.Hypergraph/HypergraphViews.cs:24-36]`

```
uint snapshotAt = 1000;  // every Handle with Index < 1000 existed at "the snapshot point"

IHypergraphQuery asOfSnapshot = Subgraph(kernel, h => h.Index < snapshotAt);
IHypergraphQuery addedSince   = Difference(kernel, asOfSnapshot);  // "what's new since the
snapshot"
```

This covers within-one-kernel-lifetime version-diffing today, with no persistence layer required. Cross-process/cross-session snapshots are M4's job (`HypergraphSnapshot`).

5.5 Semantic recall (vector search)

When to use: to find the k nearest vertices by embedding similarity — the retrieval half of a RAG-style or memory-recall loop.

`VectorIndex` is a derived structure over `PropertyKey<float[]>` data; it adds no kernel storage and swaps in a real ANN algorithm later without touching the kernel. Brute-force today (correct, simple); true ANN is gated on a real workload's scale needs. `[verified:src=src/Topos.Hypergraph/VectorIndex.cs:3-19]`

```
PropertyKey<float[]> embedding = kernel.ResolveProperty<float[]>("embedding");
var vectorIndex = new VectorIndex(kernel, embedding);

// Store embeddings on vertices as you create them.
kernel.SetProperty(embedding, turn1, [0.1f, 0.2f, 0.3f]);
kernel.SetProperty(embedding, turn2, [0.15f, 0.22f, 0.31f]);

// Recall the 5 nearest turns to a query embedding.
IReadOnlyList<(Handle Handle, float Distance)> nearest =
    vectorIndex.NearestNeighbors([0.12f, 0.21f, 0.32f], k: 5);
```

Squared Euclidean distance, ascending. Throws on `k <= 0` or embedding-dimension mismatch. `[verified:src=src/Topos.Hypergraph/VectorIndex.cs:22-41]`

Combined with feedback — recall quality improves when you record whether each recall was useful. §5.6's `EdgeStatistics` + a recall hyperedge closes that loop; see `samples/Topos.Samples.ChatMemory.RecordRecallFeedback` for the worked example.

`[verified:src=samples/Topos.Samples.ChatMemory/ChatMemory.cs:104-113]`

5.6 Learnable edges

When to use: when an edge's firing probability should learn from reward — the learning half of an adaptive loop. Generalizes RLB's `HyperEdge` theta parameters without RLB's fixed feature layout.

```
PropertyKey<LearnableEdge> edgeWeight = kernel.ResolveProperty<LearnableEdge>("edgeWeight");

// Initialize a learnable edge over 3 features (4 theta slots: 1 bias + 3 features).
Handle edge = kernel.CreateVertex(VertexRoles.Edge);
kernel.SetProperty(edgeWeight, edge, LearnableEdge.CreateUninitialized(featureCount: 3));

// Evaluate firing probability given a feature vector.
var features = new ReadOnlySpan<float>([1.0f, 0.5f, 0.2f]);
var current = kernel.TryGetProperty(edgeWeight, edge, out var w) ? w :
LearnableEdge.CreateUninitialized(3);
float p = current.Evaluate(features);   // sigmoid(theta · [1, features...])

// Reinforce: take a gradient-ascent step toward reward, write the new edge back.
var reinforced = current.Reinforce(features, reward: 1.0f, learningRate: 0.1f);
kernel.SetProperty(edgeWeight, edge, reinforced);
```

`LearnableEdge` is an immutable value type — `Reinforce` returns a *new* instance and you `SetProperty` it back over the old one. `[verified:src=src/Topos.Hypergraph/LearnableEdge.cs:3-14]`

Per-membership statistics ride alongside on the same edge:

```
PropertyKey<EdgeStatistics> stats = kernel.ResolveProperty<EdgeStatistics>("stats");
var s = kernel.TryGetProperty(stats, edge, out var existing) ? existing :
EdgeStatistics.Initial;
s = s.Observe(succeeded: true);   // EMA update
kernel.SetProperty(stats, edge, s);
```

The EMA `Observe` rule is a default, not a mandate.
`[verified:src=src/Topos.Hypergraph/EdgeStatistics.cs:3-13]`

5.7 Persistence (save / reload)

When to use: to round-trip a kernel through disk — checkpoint state, reload across sessions, or hand a graph to another process.

`HypergraphSnapshot.Save/Load` serializes topology (vertices, incidences, allocator state) **plus an explicit, caller-specified set of typed property columns**. It does not introspect property types automatically. `[verified:src=src/Topos.Hypergraph.Persistence/HypergraphSnapshot.cs:45-88]`

```
using Topos.Hypergraph.Persistence;

PropertyKey<string>  name      = kernel.ResolveProperty<string>("name");
PropertyKey<float[]> embedding = kernel.ResolveProperty<float[]>("embedding");

var columns = new IPersistedPropertyColumn[]
{
    PersistedProperty.String(name),
    PersistedProperty.Single(embedding),   // float, not double
};

using (var stream = File.Create("memory.snap"))
    HypergraphSnapshot.Save(kernel, stream, columns);

HypergraphKernel reloaded;
using (var stream = File.OpenRead("memory.snap"))
    reloaded = HypergraphSnapshot.Load(stream, columns);   // same column list — names must
match
```

What this is and isn't: `[verified:src=src/Topos.Hypergraph.Persistence/HypergraphSnapshot.cs:25-44]`

- It *is* an explicit save/load step that preserves Handle identity across reload (Invariant 1 intact: `Save` writes `NextHandleIndex`, `Load` resumes allocation after it).
- It is *not* a transparent hot/cold hybrid kernel (no auto-spill under memory pressure).
- It is *not* an LSM tree (no WAL, no compaction, no crash safety beyond a completed write).
- Built-in codecs cover `int` / `long` / `double` / `float` / `bool` / `string` / `byte` / `DateOnly`. Anything else needs `PersistedProperty.Custom<T>`.
  `[verified:src=src/Topos.Hypergraph.Persistence/PersistedProperty.cs:53-80]`

The transparent tiered version is real follow-on work, gated on a forcing workload.
`[verified:src=src/Topos.Hypergraph.Persistence/LruCache.cs:3-12]`

5.8 Directed (role-aware) traversal

When to use: when your hyperedge has direction (Anchor→Target, Speaker→Mention, Before→After) and the kernel's role-blind traversal would give the wrong answer. Requires the `Topos.Hypergraph.Knowledge` package (M9).

Recall: every algorithm on `IHypergraphQuery` (`GetBfs`, `GetShortestPath`, etc.) is **role-blind** by design — "the kernel does not judge." Two unrelated consumers (ChatMemory, NexusVerifier) plus RLB's own `ToposGraphProjection` each hand-rolled the same ~10-line role-filtered walk before M9. `DirectedBfs` / `DirectedShortestPath` / `RoleFilteredMembers` generalize that pattern into a tested package. `[verified:src=src/Topos.Hypergraph.Knowledge/DirectedTraversal.cs:3-16]`

```
using Topos.Hypergraph;
using Topos.Hypergraph.Knowledge;

public enum ChainerRole : byte { Anchor = 0, Condition = 1, Target = 2 }

IHypergraphQuery query = kernel;   // works over a kernel, a FilteredView, a UnionView — any
source

// Multi-hop: from `a`, reach everyone reachable via Anchor→Target legs.
IReadOnlyList<Handle> reachable = query.DirectedBfs(a, ChainerRole.Anchor, ChainerRole.Target);

// One-hop: every Target of `a`'s hyperedges (generalizes ChatMemory.EntitiesMentionedIn).
IReadOnlyList<Handle> targets = query.RoleFilteredMembers(a, ChainerRole.Target);

// Shortest directed path from `a` to `c`.
IReadOnlyList<Handle> path = query.DirectedShortestPath(a, c, ChainerRole.Anchor,
ChainerRole.Target);

// Adding incidences with the typed role, no manual cast.
kernel.AddIncidence(edge, target, ChainerRole.Target, ordinal: 2);
```

`DirectedBfs` follows only hyperedges where the frontier vertex holds `fromRole`, landing on that edge's `toRole` members — so `Condition` members are correctly excluded from an Anchor→Target walk even though they share the same hyperedges. `[verified:src=src/Topos.Hypergraph.Knowledge/DirectedTraversal.cs:19-51]` `[verified:src=tests/Topos.Hypergraph.Knowledge.Tests/DirectedTraversalTests.cs:38-59]`

`TRole` must be a **byte-backed** enum (`enum Foo : byte`) — a wider underlying type throws `ArgumentException`. `[verified:src=src/Topos.Hypergraph.Knowledge/RoleExtensions.cs:37-53]`

> **Don't reach for `HasCycle` for cycle detection in a directed graph.** At the kernel's role-blind layer, `HasCycle()` returns `true` almost always on any real n-ary hypergraph (three co-members are trivially "cyclic"). If you need directed cycle detection, walk with `DirectedBfs` and guard the current path yourself. `[verified:src=src/Topos.Hypergraph/IHypergraphQuery.cs:277-306]`

5.9 Ranking and structural analysis (centrality, PageRank, directed SCC)

When to use: once you have a graph of any size, three questions come up constantly — *which vertex matters most* (centrality/PageRank), and *does this directed relationship contain a cycle that shouldn't be there* (directed SCC). M11 phase 1 added all three as GDS-verified algorithms.

Centrality:

```
using Topos.Hypergraph;

IReadOnlyDictionary<Handle, int>    degree      = Centrality.Degree(kernel);
IReadOnlyDictionary<Handle, double> closeness   = Centrality.Closeness(kernel);
IReadOnlyDictionary<Handle, double> betweenness = Centrality.Betweenness(kernel);

var mostConnected = degree.OrderByDescending(kv => kv.Value).Take(5);
```

PageRank:

```
IReadOnlyDictionary<Handle, double> rank = PageRank.Compute(kernel, damping: 0.85);
var mostImportant = rank.OrderByDescending(kv => kv.Value).First();
```

Both share the same clique-expansion adjacency `Modularity` / `TriangleCount` / `LabelPropagation` use (not `GetBfs`'s member-only hop convention), and both are GDS-verified — see §4.5.

Directed SCC — catching cycles that shouldn't exist:

```
using Topos.Hypergraph.Knowledge;

public enum DerivationRole : byte { Derived = 0, Source = 1 }

// Build a directed derivation edge per (derived fact, source fact) pair.
Handle derivation = kernel.CreateVertex(VertexRoles.Edge);
kernel.AddIncidence(derivation, factA, DerivationRole.Derived, ordinal: 0);
kernel.AddIncidence(derivation, factB, DerivationRole.Source,  ordinal: 1);

IReadOnlyList<IReadOnlyList<Handle>> components =
    kernel.DirectedScc(DerivationRole.Derived, DerivationRole.Source);

// Every non-cyclic fact lands in its own singleton component — filter those out to isolate real
cycles.
var cycles = components.Where(c => c.Count > 1).ToList();
```

An agent deriving fact A from B, B from C, and C from A is circular reasoning, not three independent facts — `DirectedScc` (iterative Tarjan, GDS-verifiable via `gds.scc` over a role-projected directed graph) is the tested, generic way to catch it. Worked example (all three): `samples/Topos.Samples.ChatMemory` — `MostConnectedEntities`, `RankByImportance`, `RecordDerivation` / `DetectCircularDerivations`.

---

6. MCP server — agent tool-calling

M10 (implemented 2026-07-27). The `Topos.Hypergraph.Mcp` package exposes Topos's public API as **Model Context Protocol tools**, so any MCP-aware agent (Claude Code, Cursor, Continue, ZCode, anything speaking the 2025-11-25 MCP spec) can create vertices, build n-ary hyperedges, query traversals, and run directed/role-aware search — **without writing any C#**. Built on Microsoft's official `ModelContextProtocol` C# SDK (v1.4.1).
`[verified:src=src/Topos.Hypergraph.Mcp/Topos.Hypergraph.Mcp.csproj]`
`[verified:docs=docs/DECISIONS.md — "M10 APPROVED AND IMPLEMENTED" entry]`

6.1 What it is

A thin JSON-RPC wrapper over Topos's existing, tested primitives — **no new graph logic, no kernel changes.** Every tool method is a ~5-line call into `HypergraphKernel`, `IHypergraphQuery`, `DirectedTraversal`, or `VectorIndex`. The server stays dumb and faithful — the same "**kernel records; it does not judge**" philosophy applied to a new shape: the *agent* (layer 1) decides what role bytes mean and what cardinalities to enforce, not the server. `[verified:src=src/Topos.Hypergraph.Mcp/ToposMcpServer.cs:7-18 — the class doc]`

Three files, ~480 lines total: `[verified:src=src/Topos.Hypergraph.Mcp/ToposMcpServer.cs]`
`[verified:src=src/Topos.Hypergraph.Mcp/TypeMapping.cs]`
`[verified:src=src/Topos.Hypergraph.Mcp/Transport/StdioHost.cs]`

File	Role
`ToposMcpServer.cs`	The `[McpServerToolType]` class — all 18 `[McpServerTool]` methods.
`TypeMapping.cs`	`Handle` ↔ opaque wire string; the `PropertyValue` tagged union; result DTOs.
`Transport/StdioHost.cs`	The host entry point — wires stdio transport + tool registration.

Design (per the M10 decision, all four §5 forks resolved): stateful single-session (one `HypergraphKernel` per server process, no persistence, state lost on exit); stdio transport only; opaque-string Handles; tagged-union property values. `[verified:docs=docs/DECISIONS.md — M10 entry, all four forks]`

6.2 Wire it into an agent

The server speaks stdio — the canonical agent transport. An agent spawns it as a subprocess. Two ways to register it:

The repo's own `.mcp.json` (used by ZCode, Claude Code, etc. — copy this shape into your project's `.mcp.json`):

```
{
  "mcpServers": {
    "topos": {
      "command": "dotnet",
      "args": [
        "run",
        "--project",
        "path/to/Topos/src/Topos.Hypergraph.Mcp/Topos.Hypergraph.Mcp.csproj",
        "-c",
        "Release",
        "--no-build"
      ]
    }
  }
}
```

`[verified:src=samples/Topos.Samples.McpAgent/.mcp.json.example]`
`[verified:src=samples/Topos.Samples.McpAgent/README.md]`

Then restart your agent. It discovers the 18 tools automatically via MCP's `tools/list` and can call them via `tools/call`.

> **Build first:** `dotnet build src/Topos.Hypergraph.Mcp -c Release` from the repo root before the agent first spawns it (the `--no-build` flag assumes a built binary).
> `[verified:src=samples/Topos.Samples.McpAgent/README.md:9]`

6.3 The 18 tools

Each is a `[McpServerTool]` method on `ToposMcpServer`, wrapping the cited primitive. Tool names use `snake_case` (MCP convention); parameters are JSON.

Vertices

Tool	Wraps	Source
`create_vertex`	`HypergraphKernel.CreateVertex(VertexRoles)` — set `isEdge=true` for a hyperedge	`[verified:src=src/Topos.Hypergraph.Mcp/ToposMcpServer.cs:26-31]` `[verified:src=src/Topos.Hypergraph/HypergraphKernel.cs:51-62]`
`get_vertex`	`TryGetVertex` — returns null for unallocated handles	`[verified:src=src/Topos.Hypergraph.Mcp/ToposMcpServer.cs:33-40]`
`count_vertices`	`CountVertices`	`[verified:src=src/Topos.Hypergraph.Mcp/ToposMcpServer.cs:42-43]`
`set_vertex_status`	`SetDormant` / `Reactivate`	`[verified:src=src/Topos.Hypergraph.Mcp/ToposMcpServer.cs:45-51]` `[verified:src=src/Topos.Hypergraph/HypergraphKernel.cs:100-106]`

Incidences

Tool	Wraps	Source
`add_incidence`	`AddIncidence(source, member, byte role, ordinal)`	`[verified:src=src/Topos.Hypergraph.Mcp/ToposMcpServer.cs:55-60]` `[verified:src=src/Topos.Hypergraph/HypergraphKernel.cs:116-126]`
`get_hyperedge_vertices`	`GetHyperedgeVertices` — every incidence on a hyperedge	`[verified:src=src/Topos.Hypergraph.Mcp/ToposMcpServer.cs:62-67]`
`get_vertex_hyperedges`	`GetVertexHyperedges` — every hyperedge a vertex belongs to	`[verified:src=src/Topos.Hypergraph.Mcp/ToposMcpServer.cs:69-74]`

Typed properties (tagged union)

The wire shape is a tagged union — `value.Type` selects which field is read. **Four types are supported:** `"string"`, `"number"` (double), `"bool"`, `"embedding"` (float[]). Unknown types throw `ArgumentException`. `[verified:src=src/Topos.Hypergraph.Mcp/TypeMapping.cs:53-60]` `[verified:src=src/Topos.Hypergraph.Mcp/ToposMcpServer.cs:78-104]`

Tool	Wraps	Source
`set_property`	`SetProperty<T>` — `value` is `{Type, StringValue/NumberValue/BoolValue/EmbeddingValue}`	`[verified:src=src/Topos.Hypergraph.Mcp/ToposMcpServer.cs:78-104]` `[verified:src=src/Topos.Hypergraph/HypergraphKernel.cs:154-156]`
`get_property`	`TryGetProperty<T>` — `type` selects the pool	`[verified:src=src/Topos.Hypergraph.Mcp/ToposMcpServer.cs:106-119]`
`remove_property`	`RemoveProperty<T>`	`[verified:src=src/Topos.Hypergraph.Mcp/ToposMcpServer.cs:121-133]`

> **Type narrowness is deliberate for v1.** `int`, `long`, `DateOnly`, `byte`, and custom types aren't in the tagged union — agents overwhelmingly store names, scores, flags, and embeddings, and an agent wanting a date can store an ISO string. Adding more types is a v1.1 concern.

Kernel-level query (role-blind, topology-only)

Tool	Wraps	Source
`is_reachable`	`IHypergraphQuery.IsReachable`	`[verified:src=src/Topos.Hypergraph.Mcp/ToposMcpServer.cs:137-139]`
`shortest_path`	`GetShortestPath`	`[verified:src=src/Topos.Hypergraph.Mcp/ToposMcpServer.cs:141-146]`
`bfs`	`GetBfs`	`[verified:src=src/Topos.Hypergraph.Mcp/ToposMcpServer.cs:148-153]`
`connected_components`	`GetConnectedComponents`	`[verified:src=src/Topos.Hypergraph.Mcp/ToposMcpServer.cs:155-160]`

Knowledge (M9) — role-aware directed traversal

Tool	Wraps	Source
`directed_bfs`	`DirectedTraversal.DirectedBfs`	`[verified:src=src/Topos.Hypergraph.Mcp/ToposMcpServer.cs:164-169]` `[verified:src=src/Topos.Hypergraph.Knowledge/DirectedTraversal.cs:19-51]`
`directed_shortest_path`	`DirectedTraversal.DirectedShortestPath`	`[verified:src=src/Topos.Hypergraph.Mcp/ToposMcpServer.cs:171-176]`
`role_filtered_members`	`DirectedTraversal.RoleFilteredMembers`	`[verified:src=src/Topos.Hypergraph.Mcp/ToposMcpServer.cs:178-183]`

Semantic recall (M5)

Tool	Wraps	Source
`semantic_recall`	`VectorIndex.NearestNeighbors` — exact k-NN over a named embedding property	`[verified:src=src/Topos.Hypergraph.Mcp/ToposMcpServer.cs:187-193]` `[verified:src=src/Topos.Hypergraph/VectorIndex.cs:30-41]`

6.4 A worked example: agent builds and queries a hyperedge

This is the round-trip an agent actually performs — the same shape the test suite pins (`EndToEnd_Hyperedge_Bfs_And_RoleAware_Traversal`) and the same shape verified live over the real stdio transport in the M10 dogfooding pass.
`[verified:src=tests/Topos.Hypergraph.Mcp.Tests/ToposMcpServerTests.cs:61-99]`
`[verified:docs=docs/DECISIONS.md — M10 entry, "initialize/tools/list/tools/call JSON-RPC frames sent by hand"]`

Conceptually, the agent calls these tools in order (no C# written by the agent's author):

```
1. create_vertex()                                  → "#0"  (alice)
2. create_vertex()                                  → "#1"  (kyoto)
3. create_vertex()                                  → "#2"  (nara)
4. create_vertex()                                  → "#3"  (osaka)
5. create_vertex(isEdge=true)                       → "#4"  (the hyperedge)

6. add_incidence(source="#4", member="#0", role=0, ordinal=0)   # alice is the Speaker
7. add_incidence(source="#4", member="#1", role=1, ordinal=1)   # kyoto is Mentioned
8. add_incidence(source="#4", member="#2", role=1, ordinal=2)   # nara is Mentioned
9. add_incidence(source="#4", member="#3", role=1, ordinal=3)   # osaka is Mentioned

10. set_property(handle="#1", name="displayName",
                 value={Type="string", StringValue="Kyoto"})

11. is_reachable(from="#0", to="#3")            → true   (co-incident on #4)
12. role_filtered_members(vertex="#0", role=1)  → ["#1", "#2", "#3"]  (the three mentions)
13. directed_bfs(start="#0", fromRole=0, toRole=1)
                                                → ["#0", "#1", "#2", "#3"]  (multi-hop via
Speaker→Mention)
```

Role bytes (`0` = Speaker, `1` = Mention) are the agent's convention — the server stores them faithfully and never interprets them, exactly like the kernel. See `docs/ROLE_CONVENTIONS.md` for the byte-backed-enum pattern.

6.5 What's deliberately not exposed

Three primitives are omitted from the MCP surface, by design (`docs/MCP_SERVER_SPEC.md §4`):

Omitted	Why
`RestoreVertex`	The only sanctioned Invariant-1 bypass (snapshot reload only). Exposing it over MCP would be a footgun — an agent could create real Handle collisions. `[verified:src=src/Topos.Hypergraph/HypergraphKernel.cs:71–81]`
`AllIncidences`	An internal-iteration helper for snapshot persistence, not a query an agent needs. `[verified:src=src/Topos.Hypergraph/HypergraphKernel.cs:134–140]`
`HasCycle`	Returns `true` almost always on real n-ary hypergraphs (three co-members are trivially "cyclic") — a documented footgun at the API layer, worse over MCP where the agent can't see the warning in the XML doc. `[verified:src=src/Topos.Hypergraph/IHypergraphQuery.cs:277–306]`

If an agent needs cycle detection in a *directed* sense, it walks with `directed_bfs` and guards the current path itself — the server doesn't pretend to offer a meaningful `has_cycle` .

6.6 State lifetime and limitations

Stateful, single-session: one `HypergraphKernel` lives for the server process's lifetime (`private static readonly HypergraphKernel Kernel = new();`). Everything an agent builds is **lost when the process exits** — there's no auto-persistence in v1. Restarting the agent (which respawns the subprocess) starts a fresh, empty graph. `[verified:src=src/Topos.Hypergraph.Mcp/ToposMcpServer.cs:22]` `[verified:src=samples/Topos.Samples.McpAgent/README.md — "State lifetime" section]`

Three things v1 deliberately does not include (each gated on a real consumer needing it, same discipline the rest of the project applies to deferrals):

- **No HTTP/SSE transport** — stdio only. Remote-agent support is a v2 concern.
- **No multi-tenancy** — one server, one kernel, one agent. Multi-tenant (N agents sharing one server) is a v3 concern.
- **No auto-persistence** — the M4 `HypergraphSnapshot` is the building block; wiring lifecycle hooks to call it is v2 work, gated on a real consumer needing state to survive restarts.

The full scope, design forks, and rationale are in `docs/MCP_SERVER_SPEC.md`; the build record and the live JSON-RPC dogfooding results are in `docs/DECISIONS.md`'s "M10 APPROVED AND IMPLEMENTED" entry.

Cross-references (outside this file)

- **The full spec** (storage contract, layer architecture, roadmap, design patterns) → `docs/SPECIFICATION.md`.
- **What's locked vs. open** → `docs/SPECIFICATION.md §11` (quick-reference table) and `docs/DECISIONS.md`.
- **The MCP server's full design rationale** (the five §5 forks, the forcing-function case, the v1 scope) → `docs/MCP_SERVER_SPEC.md`. The build record and live JSON-RPC dogfooding results are in `docs/DECISIONS.md`'s "M10 APPROVED AND IMPLEMENTED" entry.
- **Role-byte conventions** (the `byte`-backed-enum pattern) → `docs/ROLE_CONVENTIONS.md`.
- **NuGet publish steps** (the gated item, MIT license decision recorded) → `docs/NUGET_PUBLISH_CHECKLIST.md`.
- **GDS-oracle setup** (the Neo4j correctness oracle behind the GDS-verified claims above) → `docs/GDS_ORACLE_SETUP.md`.